

Conditional Drifting Models for Learned Channel Simulation

Rick Fritschek and Rafael F. Schaefer
TU Dresden, Germany

Abstract—Accurate stochastic channel simulation is central to data-driven communication system design, especially when measured channels are costly to acquire repeatedly or when differentiable learned surrogates are needed inside end-to-end optimization loops. Recent work has shown that conditional diffusion models are strong channel generators, but their inference cost scales with the number of reverse denoising steps. This paper studies conditional drifting models as a low-latency alternative and evaluates them against conditional DDPM, conditional DDIM, and conditional WGAN baselines. We evaluate two drifting variants: residual drifting, which models $e = y - x$, and conditional (direct-output) drifting, which models $p(y | x)$ directly. The benchmark covers AWGN, Rayleigh fading, SSPA nonlinearity, and a simplified optical-fiber proxy channel. Both drifting variants adapt the drifting-model idea to conditional channel generation by using attraction–repulsion drift fields in their respective output spaces. We show that diffusion baselines provide the strongest SWD overall, while both drifting variants remain competitive. Moreover, a timing study shows that one-shot drifting retains microsecond-level inference while diffusion samplers remain in the millisecond regime. These results position drifting as a promising fast alternative to iterative diffusion-based channel generators.

Index Terms—learned channel models, conditional generative modeling, diffusion models, drifting models, sliced Wasserstein distance, inference latency

I. INTRODUCTION

Accurate channel models are a basic requirement for modern communication-system design. They are especially important in neural and end-to-end learned systems, where the channel appears repeatedly inside training loops, decoder adaptation procedures, and differentiable autoencoder pipelines. When a clean analytical channel model is unavailable, too restrictive, or poorly matched to measured data, generative channel surrogates become attractive because they can learn the full conditional distribution directly from samples.

This problem is not only about distributional accuracy. In many learning pipelines, the channel model is evaluated millions of times. As a result, sample quality, conditioning accuracy, computational cost, and differentiability all matter simultaneously. The question is therefore not merely whether a generative model can match the channel distribution, but whether it can do so at a cost compatible with large-scale training and deployment workflows.

Recent work on diffusion-based channel modeling has shown that conditional diffusion models can generate high-quality channel samples across standard benchmarks such as AWGN, Rayleigh, and nonlinear amplifier channels [1]. Follow-up work has further studied robustness and

speed-quality tradeoffs for diffusion-based channel generation [2], extended diffusion-based channel synthesis to high-dimensional user-specific channels [3], and proposed diffusion-driven digital-twin style generation of statistical channel state information from sensing and location information [4]. Recent 6G overview papers also identify generative and diffusion-based methods as part of the emerging AI/ML physical-layer toolbox [5], [6]. That line of work is compelling because diffusion models are flexible, robust, and often more stable than earlier adversarial approaches. However, this improvement comes with a practical systems cost: inference requires an iterative reverse denoising procedure, and the latency scales with the number of diffusion steps. This tradeoff is manageable when the channel generator is called occasionally, but it becomes more problematic when it is embedded in inner loops of iterative training or repeated Monte Carlo evaluation. This latency bottleneck has motivated several acceleration strategies for diffusion, including skipped or reduced-step sampling as in DDIM, as well as one-step or few-step alternatives such as consistency models and distillation-based approaches that compress multi-step samplers into substantially shorter generation paths [7]–[9]. This motivates the study of one-shot conditional generators for learned channel simulation. Among recently proposed generative paradigms, drifting models [10] are particularly interesting because they aim to learn the transport from a simple latent source to the target data distribution directly in the network parameters, rather than unfold that transport as a multi-step sampling trajectory at inference time. Intuitively, diffusion spends computation at test time by progressively transforming noise into data through a reverse-time chain. Drifting instead spends computation during training: generated samples are repeatedly nudged toward the data manifold by a drift field, and the neural generator is updated so that future samples move closer to the desired distribution in a single forward pass. Once trained, sampling is therefore naturally one-shot. This perspective is also consistent with recent geometric analyses of generative dynamics that relate diffusion-like behavior to autonomous vector-field structure and marginalized noise geometry [11].

That distinction makes drifting conceptually appealing for channel modeling. In a communication setting, the generator must be conditional on the transmitted symbol x , and it must capture not just a marginal sample cloud, but the conditional law $p(y | x)$. Because drifting was introduced very recently, no widely adopted communication-channel baseline has yet

emerged. This motivates reporting not only benchmark outcomes, but also the concrete conditional adaptation needed to integrate drifting into an existing channel-learning pipeline.

The emphasis of this study is on generator-level distributional accuracy and computational tradeoffs rather than on downstream symbol-error-rate curves.

The main contributions are summarized as follows.

- We formulate conditional drifting generators for channel simulation in both residual form, using $e = y - x$, and direct-output form, modeling $p(y | x)$ directly.
- We provide a practical drifting objective based on attraction and repulsion in the chosen target space, which is straightforward to integrate into an existing PyTorch channel-learning pipeline.
- We compare the resulting drifting variants against conditional DDPM, conditional DDIM, and conditional WGAN models on AWGN, Rayleigh, SSPA, and optical-fiber proxy channels.
- We quantify the accuracy–latency tradeoff and show that one-shot drifting provides large inference-time savings while remaining competitive in distributional accuracy.

Source code is available online¹.

II. PROBLEM FORMULATION AND DRIFTING BACKGROUND

A. What drifting changes conceptually

Because drifting is very recent and remains less familiar than diffusion, we briefly summarize its generative viewpoint. Following [10], let f_θ map a latent variable $z \sim p_z$ to a sample $u = f_\theta(z)$, and let

$$q_\theta = (f_\theta)_\# p_z \quad (1)$$

denote the pushforward distribution induced by the generator. The training objective is no longer expressed as a likelihood surrogate or as a denoising score-matching loss. Instead, the paper introduces a *drifting field* that tells us how samples drawn from the current model distribution should move so that the model distribution approaches the target distribution p .

At a high level, the desired property is an equilibrium condition: if the model already matches the target distribution, then the drift should vanish. Using $V_{p,q}(u)$ to denote the field evaluated at a sample u under target distribution p and model distribution q , the fixed-point intuition can be summarized as

$$q = p \implies V_{p,q}(u) = 0, \quad (2)$$

which is the sense in which matched distributions form an equilibrium of the training dynamics [10].

This makes the training logic very different from diffusion. In diffusion, one learns a denoiser and then performs iterative refinement at inference time. In drifting, one instead constructs improved targets during training by moving current samples along the field $V_{p,q}$, and then regresses the generator onto those

improved targets. As a result, the iterative correction happens during optimization rather than at test time.

The one-step view can be written as follows. Starting from a generated sample $u = f_\theta(z)$, define a drifted target

$$\tilde{u} = \text{stopgrad}(u + \eta V_{p,q_\theta}(u)), \quad (3)$$

and optimize the generator by

$$\mathcal{L}_{\text{drift}} = \mathbb{E}_{z \sim p_z} [\|f_\theta(z) - \tilde{u}\|_2^2]. \quad (4)$$

The stop-gradient is essential here: it makes the drifted sample behave like a fixed target for the current optimization step. Repeated training steps therefore evolve the pushforward q_θ toward the target distribution. This is also the cleanest way to see why drifting is naturally one-shot at inference: once the generator has learned to emit samples near equilibrium, sampling only requires drawing z and evaluating $f_\theta(z)$, with no reverse-time chain.

Fig. 1 gives an actual toy particle simulation in the spirit of the particle-based demonstrations used in [10]. For two fixed channel inputs, we initialize mismatched conditional residual particle clouds and then evolve them under the same attraction–repulsion drift rule used in our channel benchmark. The figure therefore illustrates real drift dynamics rather than a hand-drawn schematic, while still serving only as a toy analogue of the full learned generator. To show that the same idea can train an explicit parameterized generator, Fig. 2 uses a tiny conditional MLP trained with the drifting objective on the same toy target.

B. Conditional channel simulation

We now specialize the generic drifting view to learned conditional channel simulation. Let $x \in \mathbb{R}^n$ denote the transmitted channel input and $y \in \mathbb{R}^n$ the channel output. The objective is to learn a conditional generator that reproduces the channel law given x . In this setting, conditioning enters through the transmitted symbol, while stochasticity is carried by a latent variable $z \sim \mathcal{N}(0, I)$.

Two target parameterizations are useful in the experiments. The first is the direct channel-output formulation, in which the generator approximates $p(y | x)$ and predicts $\hat{y} = g_\theta(x, z)$. The second is the residual formulation, in which

$$e = y - x \quad (5)$$

is modeled instead, so that the generator approximates $p(e | x)$ and the channel output is reconstructed as

$$\hat{y} = x + \hat{e}. \quad (6)$$

The direct formulation corresponds to learning the channel law itself, whereas the residual formulation injects the inductive bias that the channel output can be understood as the transmitted symbol plus a structured perturbation. As discussed later in the experiments, the direct formulation is a natural continuation to direct-output diffusion and WGAN baselines, while the residual formulation is easier to optimize but can also fall short in non-linear channels.

¹https://github.com/Fritschek/conditional_drifting_models.

TABLE I
TWO TARGET PARAMETERIZATIONS FOR CONDITIONAL DRIFTING.

Mode	Output of g_θ	Target object	Short interpretation
Direct	$\hat{y}$	Learn $p(y x)$	Learns the channel law directly
Residual	$\hat{e}$, then $\hat{y} = x + \hat{e}$	Learn $p(e x)$ with $e = y - x$	Learns the channel perturbation

C. Conditional drifting model

We use a practical conditional adaptation inspired by [10]. To avoid confusion with the communication input symbol x , we use u above for the generic sample variable in the abstract formulation and reserve x below for the transmitted symbol. In our implementation, drifting can be applied either in direct-output space or in residual space. In direct mode, the generator predicts $\hat{y} = g_\theta(x, z)$; in residual mode, it predicts $\hat{e} = g_\theta(x, z)$ with reconstruction $\hat{y} = x + \hat{e}$, where

$$z \sim \mathcal{N}(0, I). \quad (7)$$

In both cases, g_θ is implemented as a two-hidden-layer multi-layer perceptron with hidden width 128, latent dimension 16, and SiLU activations:

$$[x, z] \rightarrow \text{Linear} \rightarrow \text{SiLU} \rightarrow \text{Linear} \rightarrow \text{SiLU} \rightarrow \text{Linear}. \quad (8)$$

The training target is formed from an attraction–repulsion drift field in the chosen target space. This is a concrete channel-specific instantiation of the generic field $V_{p,q}$. Given generated samples $G = \{g_i\}$ and positive samples $P = \{p_j\}$, we compute

$$V(g_i) = \left(\sum_j w_{ij}^{(P)} p_j - g_i \right) - \lambda \left(\sum_k w_{ik}^{(G)} g_k - g_i \right), \quad (9)$$

where $w^{(P)}$ and $w^{(G)}$ are row-normalized RBF kernel weights. The first term attracts each generated sample toward a local barycenter of true samples in the chosen target space, and the second term repels it from nearby generated samples to reduce collapse. This yields a detached target

$$\tilde{g}_i = \text{stopgrad}(g_i + \eta V(g_i)), \quad (10)$$

and the generator is optimized by

$$\mathcal{L}_{\text{drift}} = \frac{1}{N} \sum_i \|g_i - \tilde{g}_i\|_2^2. \quad (11)$$

In other words, the generic update $u \mapsto u + \eta V_{p,q}(u)$ from the drifting formulation is realized here with a kernel-based drift field estimated directly from minibatches of true and generated samples in the selected target space.

In the simulations, the drift scale is $\eta = 1.0$, the repulsive weight is $\lambda = 1.0$, the minimum kernel bandwidth is 0.2, and the maximum drift norm is 2.0. These settings are used throughout the drifting benchmark and gave stable behavior across the channel families considered here.

The major operational advantage is straightforward: once training is complete, sampling requires only a single evaluation

of $g_\theta(x, z)$. In residual mode, this produces a channel residual sample $\hat{e}$ followed by reconstruction $\hat{y} = x + \hat{e}$. In direct mode, it produces $\hat{y}$ directly. There is no reverse chain, no ODE solver, and no per-sample iterative schedule at inference.

III. EXPERIMENTAL SETUP

A. Channel models

We simulate four channel families. All channels are implemented on real-valued vectors. For AWGN and Rayleigh, we use real-valued channel inputs and output vectors of length 7. For SSPA and OptFib, the real-valued signal is interpreted in paired I/Q coordinates, with dimensions 8 and 2, respectively. Noise is independent across real components unless the channel induces coupling.

AWGN:

$$y = x + n, \quad n \sim \mathcal{N}(0, \sigma^2 I). \quad (12)$$

This is the standard additive white Gaussian noise reference channel and serves as the simplest near-identity baseline.

Rayleigh: the transmitted symbol is multiplied elementwise by a random fading amplitude and then corrupted by additive Gaussian noise:

$$y = h \odot x + n, \quad (13)$$

where $\odot$ denotes elementwise multiplication,

$$h_k = \frac{1}{\sqrt{2}} \sqrt{u_k^2 + v_k^2}, \quad u_k, v_k \stackrel{\text{i.i.d.}}{\sim} \mathcal{N}(0, 1), \quad (14)$$

and $n \sim \mathcal{N}(0, \sigma^2 I)$.

Each real component is therefore scaled by an independent Rayleigh-distributed magnitude before noise is added. This channel is multiplicative rather than additive and is correspondingly less aligned with a pure residual perturbation view. As implemented here, Rayleigh is also a real-valued vector channel rather than a paired-I/Q complex channel. The fading amplitudes are latent channel randomness and are not supplied as additional conditioning variables.

SSPA: the input is passed through a smooth solid-state power amplifier nonlinearity before additive noise is applied. In this case, the real-valued representation is interpreted as paired I/Q components. Let

$$r = \sqrt{x_I^2 + x_Q^2} \quad (15)$$

denote the input amplitude. The amplitude-dependent gain used in the benchmark is

$$a(r) = \frac{G}{\left(1 + \left(\frac{Gr}{A_0}\right)^{2p}\right)^{1/(2p)}}, \quad (16)$$

with default parameters $p = 3$, $A_0 = 1.5$, and $G = 5.0$. Thus the noiseless output amplitude is $a(r)r$, and the noiseless amplifier output is $\tilde{x} = a(r)x$. and the observed channel output is

$$y = \tilde{x} + n, \quad n \sim \mathcal{N}\left(0, \frac{\sigma^2}{2} I\right). \quad (17)$$

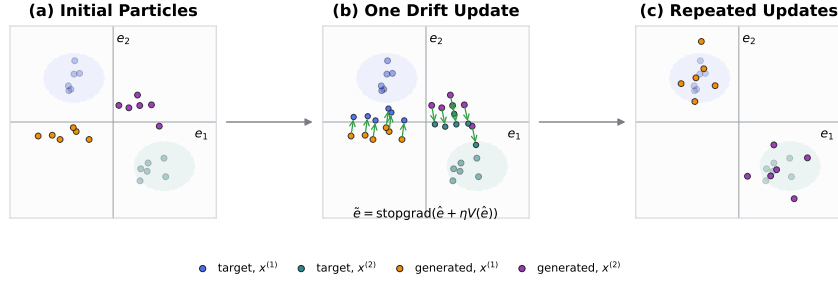


Fig. 1. Toy conditional particle simulation in residual space. Two channel inputs induce different target residual distributions. Starting from mismatched generated particles, the middle panel applies one attraction–repulsion drift update $\tilde{e} = \text{stopgrad}(\hat{e} + \eta V(\hat{e}))$, and the last panel shows the particles after repeated updates.

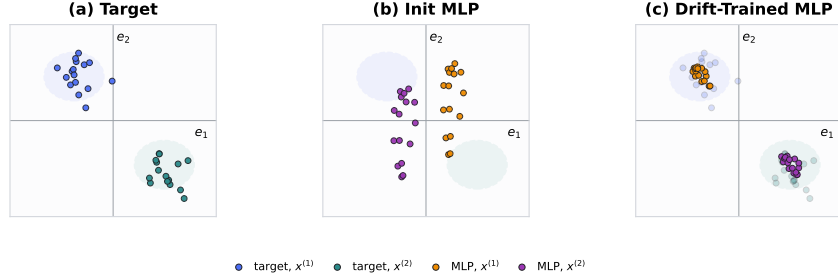


Fig. 2. Toy conditional MLP trained with the drifting objective. The left panel shows target conditional residual samples, the middle panel shows the randomly initialized generator, and the right panel shows the same generator after drifting training.

This channel introduces amplitude-dependent nonlinear distortion while preserving the input direction in the I/Q plane.

OptFib: We include a simplified memoryless optical-fiber proxy channel following [12] for the choice of parameters. As in SSPA, the real-valued state is interpreted as an I/Q pair. Let the state after propagation step k be $x^{(k)} = [x_I^{(k)}, x_Q^{(k)}]^\top$. At each step, the instantaneous nonlinear phase is

$$\phi^{(k)} = \gamma \left(\left(x_I^{(k)} \right)^2 + \left(x_Q^{(k)} \right)^2 \right) \frac{L}{K_{\text{step}}}. \quad (18)$$

The deterministic Kerr-style rotation is

$$\begin{aligned} \tilde{x}_I^{(k+1)} &= x_I^{(k)} \cos \phi^{(k)} - x_Q^{(k)} \sin \phi^{(k)}, \\ \tilde{x}_Q^{(k+1)} &= x_I^{(k)} \sin \phi^{(k)} + x_Q^{(k)} \cos \phi^{(k)}, \end{aligned} \quad (19)$$

followed by additive Gaussian perturbations at every step:

$$\begin{aligned} x_I^{(k+1)} &= \tilde{x}_I^{(k+1)} + \sigma_n \xi_I^{(k)}, \\ x_Q^{(k+1)} &= \tilde{x}_Q^{(k+1)} + \sigma_n \xi_Q^{(k)}, \end{aligned} \quad \xi_I^{(k)}, \xi_Q^{(k)} \stackrel{\text{i.i.d.}}{\sim} \mathcal{N}(0, 1). \quad (20)$$

When that optical-channel parameterization is used,

$$\sigma_n = \sqrt{\frac{10^{(P_n - 30)/10}}{2K_{\text{step}}}}, \quad (21)$$

with P_n measured in dBm.

B. Training protocol

This benchmark protocol closely follows the setup in [2].

a) Channel settings.: For AWGN and Rayleigh, the setting is $n = 7$, while for SSPA it is $n = 8$. The corresponding $(E_b/N_0, R)$ pairs are (5 dB, 4/7) for AWGN, (12 dB, 4/7) for Rayleigh, and (8 dB, 6/8) for SSPA, yielding noise standard deviations 0.5260, 0.2350, and 0.3251, respectively. For OptFib, the channel parameters are $L = 5000$, $\gamma = 1.27$, $K_{\text{step}} = 50$, and $P_n = -21.3$ dBm.

b) Data and optimization settings.: For AWGN and Rayleigh, we use training dataset size 10^7 , batch size 5000, and 30 epochs for drifting, diffusion, and WGAN. For SSPA, we use training dataset size 10^7 , batch size 4096, and 160 epochs for all three method families. For these three channels, we evaluate each method on 10^6 samples and use 128 SWD projections. OptFib is benchmarked separately with training dataset size 120,000, batch size 512, 60 training epochs, evaluation size 100,000, and 256 SWD projections.

c) Model settings.: Drifting uses a conditional MLP with latent dimension 16, hidden width 128, two hidden layers, SiLU activations, and Adam with learning rate 10^{-3} . The drifting objective uses drift scale 1.0, median-heuristic bandwidth selection, minimum bandwidth 0.2, maximum drift norm 2.0, and repulsive weight 1.0. We train both parameterizations: direct drifting with target y and residual drifting with target $e = y - x$.

For AWGN, Rayleigh, and SSPA, diffusion models $p(y | x)$ directly with v -prediction, cosine-ZF variance schedule, 100 diffusion steps, and EMA decay 0.9. Hidden widths are 110 (AWGN), 128 (Rayleigh), and 110 (SSPA). AWGN and Rayleigh use a two-stage learning-rate schedule (10 epochs at

TABLE II
MODEL PARAMETER COUNTS IN THE BENCHMARK SETUP. COUNTS REFER TO INFERENCE-TIME NETWORKS. DDPM AND DDIM SHARE THE SAME DENOISER WEIGHTS. WGAN ROWS REPORT GENERATOR-SIDE PARAMETERS.

Model	AWGN	Rayleigh / SSPA	OptFib
Drifting (direct / residual)	20,487	20,487 / 20,744	19,202
Diffusion denoiser (DDPM / DDIM)	59,847	74,247 / 60,178	72,322
WGAN generator	19,335	71,431 / 72,200	17,410

10^{-3} , then 20 epochs at 10^{-4}), while SSPA uses a constant 10^{-4} for 160 epochs. We report DDPM and DDIM with 100, 50, 20, and 10 denoising steps. For OptFib, diffusion uses residual modeling with ϵ -prediction, standard cosine schedule, 100 steps, hidden width 128, learning rate 10^{-3} , and EMA decay 0.995.

WGAN uses a conditional Wasserstein objective with RM-Sprop at 10^{-4} for generator and critic, five critic updates per generator update, and weight clipping at 0.01. Hidden widths are 128 (AWGN) and 256 (Rayleigh/SSPA). OptFib uses width 128. In the current benchmark implementation, WGAN is reported with residual-space SWD.

To compare distributional accuracy and latency with model capacity, Table II reports inference-network parameter counts used in this benchmark.

d) *Seeding and reporting.*: All benchmark results are reported as mean $\pm$ standard deviation over ten seeds. Each run uses deterministic initialization to reduce run-to-run variance unrelated to model stochasticity.

C. Evaluation metric

We focus on generator-level distributional accuracy and timing rather than full end-to-end communication metrics such as SER or E_b/N_0 curves, to reduce encoder-decoder dependency in the comparison. The primary metric is sliced Wasserstein distance (SWD), computed in the target space used by each benchmark path. For direct-output models, we report $\text{SWD}(\{\hat{y}_i\}, \{y_i\})$. For residual models, we report $\text{SWD}(\{\hat{e}_i\}, \{e_i\})$. Given true samples $P = \{v_i\}_{i=1}^N$ and generated samples $Q = \{\hat{v}_i\}_{i=1}^N$, SWD is estimated by drawing random unit directions θ_m , projecting both clouds onto those directions, computing the one-dimensional Wasserstein distance for each projection, and averaging over projections:

$$\text{SWD}(P, Q) = \frac{1}{M} \sum_{m=1}^M W_1(\{\langle \theta_m, v_i \rangle\}_{i=1}^N, \{\langle \theta_m, \hat{v}_i \rangle\}_{i=1}^N).$$

Thus SWD compares the generated and true sample clouds through many one-dimensional views and captures geometric mismatch more faithfully than a purely bin-based marginal comparison. In our experiments, SWD is estimated using 128 random one-dimensional projections on AWGN, Rayleigh, and SSPA, and 256 projections on the separate OptFib benchmark. SWD is used primarily for comparability with prior diffusion-channel benchmarks [1], [2]. Compared with histogram-based L_1 distance, it also better captures sample geometry in $\mathbb{R}^n$.

IV. RESULTS

A. Main SWD benchmark across channels

Table III reports the SWD benchmark across AWGN, Rayleigh, SSPA, and OptFib. Direct output-space SWD is used throughout the table except for the additional residual-native row “Drifting (res., e)”, which reports residual-space SWD for the residual parameterization. DDIM-100 achieves the lowest direct-space SWD on AWGN, Rayleigh, and SSPA, while DDPM is strongest on OptFib. Among the one-shot generators under direct-space evaluation, direct-output drifting is consistently strongest across all four channels, whereas residual drifting degrades substantially once judged against the full $p(y | x)$ relation. The extra residual-space row makes clear that residual drifting remains strong in its native target space. Reduced-step DDIM variants degrade systematically as the number of sampling steps is reduced. Table IV summarizes projected CUDA training and inference timing on an NVIDIA GeForce RTX 5060 Ti GPU. Training hours use the same 2% extrapolation protocol for each channel and method. Inference values are means over seven CUDA-synchronized repeats with batch size 512 and 40 batches per repeat; the largest relative standard deviation is 1.4%. The main timing result is that one-shot generators operate at microsecond-level inference, whereas diffusion samplers remain in the millisecond regime, yielding a substantial latency gap across all channels.

B. Direct-space check for residual drifting and WGAN

To quantify the effect of metric space, we additionally compare residual drifting and WGAN under both direct y -space SWD and residual-space SWD in Table V. WGAN changes little between the two metrics, whereas residual drifting degrades substantially in direct y -space, especially on Rayleigh, SSPA, and OptFib. This indicates that residual drifting is well aligned with residual modeling, but that its advantage does not transfer directly to full output-space matching.

V. CONCLUSION

This paper presented a comprehensive experimental study of conditional drifting models for learned channel simulation and compared them against conditional diffusion and WGAN baselines on AWGN, Rayleigh, SSPA, and optical-fiber proxy channels. The results show a clear tradeoff between fidelity and latency. Diffusion provides the strongest distributional accuracy overall, while drifting remains competitive and retains the one-shot advantage with substantially lower inference cost. Timing results show a persistent microsecond-versus-millisecond inference gap in favor of one-shot generators. Under direct output-space evaluation, direct-output drifting is the strongest one-shot generator across the main benchmark, whereas the additional residual-space row and the direct-vs-residual side check show that residual drifting is highly specialized to residual matching rather than full output-space fidelity. These observations suggest that drifting is a practically relevant direction for learned channel simulation, especially when inference latency matters. Future work should include broader channel families, such as MIMO and more realistic

TABLE III
TEN-SEED SWD BENCHMARK ACROSS ALL CHANNELS. LOWER IS BETTER. DIRECT OUTPUT-SPACE SWD IS REPORTED UNLESS THE ROW LABEL EXPLICITLY INDICATES RESIDUAL-SPACE SWD.

Method	AWGN	Rayleigh	SSPA	OptFib
<i>One-shot generators</i>				
Drifting (dir.)	0.0100 ± 0.0007	0.0085 ± 0.0012	0.0060 ± 0.0008	0.0275 ± 0.0104
Drifting (res., y)	0.0670 ± 0.0138	0.3681 ± 0.0031	0.4812 ± 0.0022	0.3864 ± 0.0211
Drifting (res., e)	0.0097 ± 0.0009	0.0062 ± 0.0009	0.0130 ± 0.0003	0.0463 ± 0.0082
Paper WGAN	0.0198 ± 0.0050	0.0171 ± 0.0050	0.0287 ± 0.0156	0.0414 ± 0.0179
<i>Diffusion samplers</i>				
DDPM	0.0070 ± 0.0004	0.0079 ± 0.0008	0.0032 ± 0.0005	0.0088 ± 0.0019
DDIM-100	0.0042 ± 0.0006	0.0044 ± 0.0007	0.0024 ± 0.0004	0.0584 ± 0.0148
DDIM-50	0.0064 ± 0.0004	0.0066 ± 0.0010	0.0030 ± 0.0005	—
DDIM-20	0.0143 ± 0.0003	0.0146 ± 0.0014	0.0056 ± 0.0006	—
DDIM-10	0.0264 ± 0.0004	0.0271 ± 0.0013	0.0092 ± 0.0008	—

TABLE IV
PROJECTED TRAINING AND INFERENCE TIMING ON AN NVIDIA GeForce RTX 5060 Ti GPU. TRAINING HOURS ARE EXTRAPOLATED FROM A 2% TIMING RUN; INFERENCE REPORTS MEAN TIME PER SAMPLE OVER SEVEN TIMED REPEATS.

Method	AWGN			Rayleigh			SSPA			OptFib		
	Train [h]	Inf.	Total [h]	Train [h]	Inf.	Total [h]	Train [h]	Inf.	Total [h]	Train [h]	Inf.	Total [h]
<i>One-shot generators</i>												
Drifting (res.)	0.733	0.405 us	0.734	0.730	0.407 us	0.731	3.197	0.424 us	3.198	0.054	0.410 us	0.054
Drifting (dir.)	0.724	0.381 us	0.725	0.725	0.382 us	0.726	3.183	0.396 us	3.184	0.049	0.382 us	0.049
Paper WGAN	0.206	0.393 us	0.207	0.202	0.386 us	0.203	1.352	0.408 us	1.354	0.236	0.382 us	0.236
<i>Diffusion samplers</i>												
DDPM	0.049	0.132 ms	0.416	0.050	0.134 ms	0.422	0.333	0.139 ms	0.719	0.048	0.112 ms	0.051
DDIM-100	0.049	0.114 ms	0.366	0.050	0.115 ms	0.371	0.333	0.120 ms	0.666	0.048	0.107 ms	0.051
DDIM-50	0.049	0.057 ms	0.208	0.050	0.058 ms	0.211	0.333	0.060 ms	0.500	0.048	0.054 ms	0.050
DDIM-20	0.049	0.023 ms	0.113	0.050	0.023 ms	0.115	0.333	0.024 ms	0.400	0.048	0.022 ms	0.049
DDIM-10	0.049	0.012 ms	0.081	0.050	0.012 ms	0.083	0.333	0.012 ms	0.367	0.048	0.011 ms	0.048

TABLE V
DIRECT- AND RESIDUAL-SPACE SWD FOR RESIDUAL DRIFTING AND WGAN. VALUES ARE TEN-SEED MEAN AND STANDARD DEVIATION; LOWER IS BETTER.

Method	Direct SWD	Residual SWD
<i>AWGN</i>		
Drifting (res.)	0.0670 ± 0.0138	0.0103 ± 0.0010
WGAN	0.0198 ± 0.0050	0.0195 ± 0.0051
<i>Rayleigh</i>		
Drifting (res.)	0.3681 ± 0.0031	0.0063 ± 0.0011
WGAN	0.0171 ± 0.0050	0.0170 ± 0.0050
<i>SSPA</i>		
Drifting (res.)	0.4812 ± 0.0022	0.0128 ± 0.0007
WGAN	0.0287 ± 0.0156	0.0269 ± 0.0162
<i>OptFib</i>		
Drifting (res.)	0.3864 ± 0.0211	0.0477 ± 0.0059
WGAN	0.0414 ± 0.0179	0.0424 ± 0.0181

optical or memory channels, together with downstream communication metrics.

REFERENCES

- [1] M. Kim, R. Fritschek, and R. F. Schaefer, “Diffusion models for accurate channel distribution generation,” *arXiv preprint arXiv:2309.10505*, 2023.
- [2] —, “Robust generation of channel distributions with diffusion models,” in *IEEE International Conference on Communications (ICC)*, 2024.
- [3] T. Lee, J. Park, H. Kim, and J. G. Andrews, “Generating high dimensional user-specific wireless channels using diffusion models,” *arXiv preprint arXiv:2409.03924*, 2024.
- [4] X. Gong, X. Liu, A. A. Lu, X. Gao, X. G. Xia, C.-X. Wang, and X. You, “Digital twin of channel: Diffusion model for sensing-assisted statistical channel state information generation,” *IEEE Transactions on Wireless Communications*, vol. 24, no. 5, pp. 3805–3821, 2025.
- [5] B. Banerjee *et al.*, “6G PHY: Insights from 6G-ANNA research initiative,” *IEEE Open Journal of the Communications Society*, vol. 7, pp. 1210–1234, 2026.
- [6] A. Roessler *et al.*, “AI/ML-driven 6G network solutions with energy efficiency considerations,” *IEEE Access*, vol. 14, pp. 39 617–39 645, 2026.
- [7] J. Song, C. Meng, and S. Ermon, “Denoising diffusion implicit models,” in *International Conference on Learning Representations*, 2021.
- [8] Y. Song, P. Dhariwal, M. Chen, and I. Sutskever, “Consistency models,” in *Proceedings of the 40th International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 202. PMLR, 2023, pp. 32 211–32 252. [Online]. Available: <https://proceedings.mlr.press/v202/song23a.html>
- [9] T. Yin, M. Gharbi, R. Zhang, E. Shechtman, F. Durand, W. T. Freeman, and T. Park, “One-step diffusion with distribution matching distillation,” in *CVPR*, 2024.
- [10] M. Deng, H. Li, T. Li, Y. Du, and K. He, “Generative modeling via drifting,” *arXiv preprint arXiv:2602.04770*, 2026.
- [11] S. Sahraee-Ardakan, M. Delbracio, and P. Milanfar, “The geometry of noise: Why diffusion models don’t need noise conditioning,” *arXiv preprint arXiv:2602.18428*, 2026.
- [12] S. Li, C. Häger, N. Garcia, and H. Wymeersch, “Achievable information rates for nonlinear fiber communication via End-to-End autoencoder learning,” in *Proc. European Conference on Optical Communication (ECOC)*, Rome, Italy, Sep. 2018, pp. 1–3.